LAPORAN PRAKTIKUM
APLIKASI WEB

OLEH: AHMAD EDIWAN GHANI (24051130036)

MODUL 5

TOPIK:

Penerapan regresi pada program python



TABLE OF CONTENTS

MODUL 1 1

A. Penjelasan Tugas Praktikum 3

B. Langkah-langkah dan Screenshot 3

C. Kendala yang Dialami 16

D. Kesimpulan 16

A. Penjelasan Tugas Praktikum

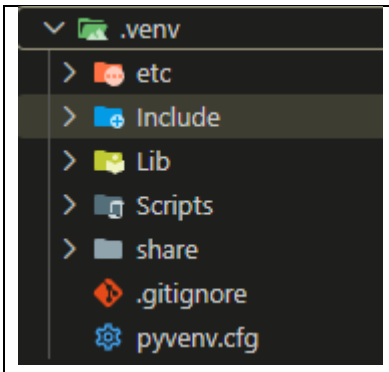
Pada praktikum minggu ini, mahasiswa mempelajari penerapan regresi linear menggunakan Python untuk membuat prediksi pada dataset. Pemrosesan data dan pelatihan model dilakukan di Google Colab, lalu hasilnya disimpan dalam bentuk file Pickle (.pkl). File Pickle ini kemudian diunggah dan di-load ke dalam aplikasi Streamlit untuk menjalankan fungsi prediksi.

B. Langkah-langkah dan Screenshot

1. Unduh dataset. Saya memakai dataset aktivitas jalan kaki dan kalori yang terbakar. Fitness Track Daily Activity Dataset
2. Pastikan bahasa Python sudah terinstall di perangkat.
3. Membuat virtual environment python pada direktori project.
   Perintah di powershell:

```
Python -m venv .venv
```

Screenshot:



4. Mengaktifkan virtual environtment .venv yang sudah dibuat.
   Perintah di powershell di folder project:

```
./.venv/Script/activate
```

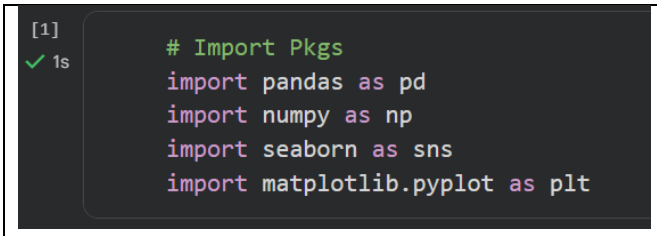
Screenshot:



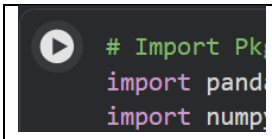
5. Membuka google colab. Menambahkan sel kode dan Import beberapa library yang dibutuhkan seperti pandas untuk mengolah data, numpy untuk komputasi numerik, seaborn dan matplotlib.pyplot untuk visualisasi data.
   Kode:

```
# Import Pkgs
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

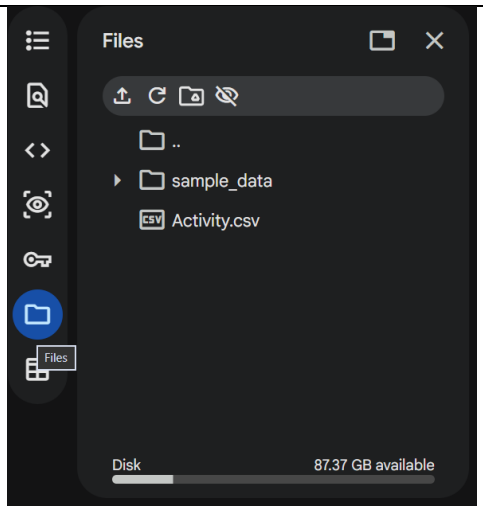
   Screenshot:



6. Menjalankan sel dengan klik tombol di sebelah kiri sel.
   Screenshot:



7. Pada menu sebelah kiri google colab, klik menu file dan upload file dataset di situ.
   Screenshot:



#Load dataset

8. Membaca dataset dan ambil kolom ”Steps” dan ”Calories_Burned” untuk digunakan pada linear regresi sederhana. Melakukan filter data dengan syarat ”Steps” > 500 dan ”Calories_Burned” > 500 sebab pada dataset kalori terbakar ditentukan oleh banyak aspek tidak hanya ”Steps”.
   - Steps > 500: Mengambil data dari individu yang benar-benar melakukan aktivitas berjalan secara aktif.
   - Calories_Burned > 500: Mengeliminasi data rusak/anomali akibat perangkat mati atau tidak dipakai

   Kode program:

```
# read the csv file, only Steps and Calories_Burned column
step_calor_df = pd.read_csv('Activity.csv',
usecols=['Steps', 'Calories_Burned'])
```

```
step_calor_df = step_calor_df[
    (step_calor_df['Steps'] > 500) &
(step_calor_df['Calories_Burned'] > 500)
]
```

```
step_calor_df = step_calor_df.reset_index(drop=True)
```

Screenshot:



9. Cek apakah masih ada data null di kedua kolom tersebut.
   Kode:

```
# check if there are any Null values
step_calor_df.isnull().sum()

# Check the dataframe info
step_calor_df.info()

# Statistical summary of the dataframe
step_calor_df.describe()
```

Screenshot:

```
# check if there are any Null values
step_calor_df.isnull().sum()
```

```
...
                    0
Steps               0
Calories_Burned     0

dtype: int64
```

```
# Check the dataframe info
step_calor_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 841 entries, 0 to 840
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Steps            841 non-null    int64
 1   Calories_Burned  841 non-null    int64
dtypes: int64(2)
memory usage: 13.3 KB
```

```
# Statistical summary of the dataframe
step_calor_df.describe()
```

	Steps	Calories_Burned
count	841.000000	841.000000
mean	8532.586207	2382.510107
std	4617.189484	692.414269
min	590.000000	741.000000
25%	5135.000000	1878.000000
50%	8199.000000	2235.000000
75%	11177.000000	2846.000000
max	36019.000000	4900.000000

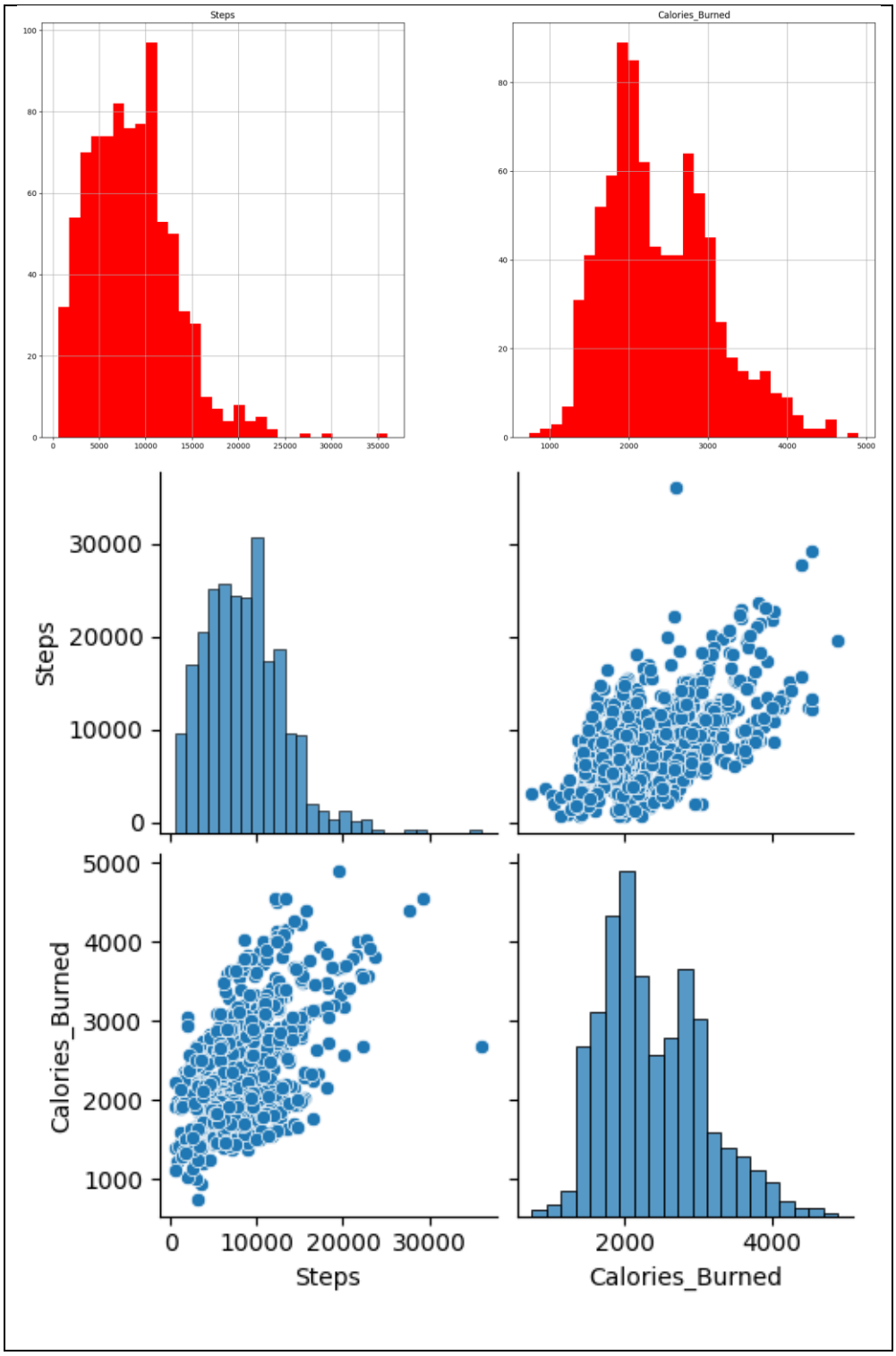
10. Membuat visualisasi persebaran data dengan grafik histogram dan pairplot.

Kode program:

```
# Histogram Plotting (Data Distribution)
step_calor_df.hist(bins = 30, figsize = (20,10),
color = 'r')

# Plot Pairplot (Variables Relationship)
sns.pairplot(step_calor_df)
```

Screenshot:



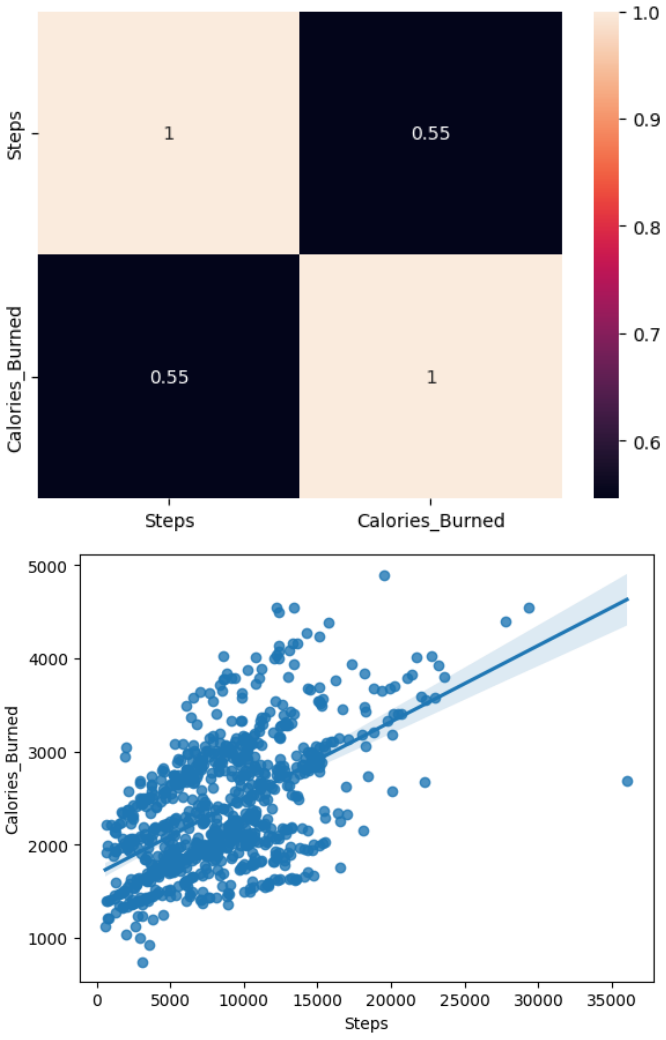
11. Mengecek korelasi antara ”Steps” dan ”Calories_Burnes” dengan bitmap korelasi matriks dan plot regresi. Hasilnya adalah ”Steps” dan ”Calories_Burnes” memiliki korelasi 0.55 atau sedang.

Kode program:

```
# Correlation Matrix
corr_matrix = step_calor_df.corr()
sns.heatmap(corr_matrix, annot = True)
plt.show()

# Regression Plot with Seaborn (straight line
fit between "salary" and "years of experience")
sns.regplot(x='Steps',y='Calories_Burned',data=s
tep_calor_df)
```

Screenshot:

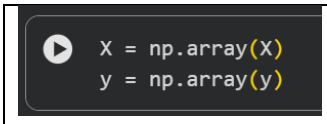


12. Menyiapkan data.

Kode program mengubah data tiap kolom menjadi susunan array:

```
X = np.array(X)
y = np.array(y)
```

Screenshot:

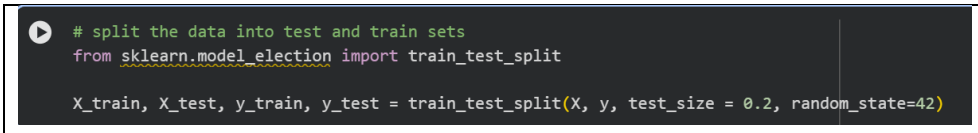


Kode program membagi data untuk ditest dan dilatih:

```
# split the data into test and train sets
from sklearn.model_selection import
train_test_split

X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size = 0.2,
random_state=42)
```

Screenshot:



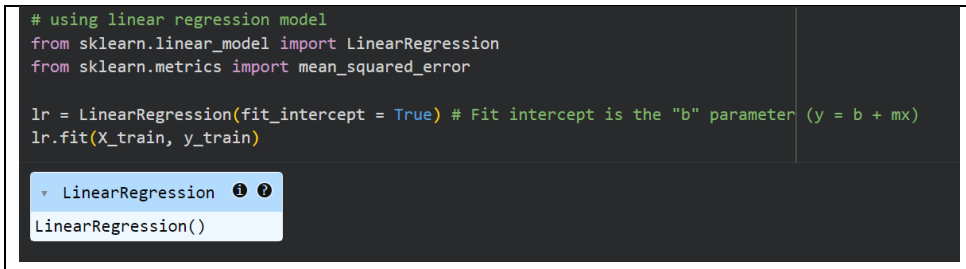
13. Melatih model regresi linear

Kode program membagi data untuk ditest dan dilatih:

```
# using linear regression model
from sklearn.linear_model import
LinearRegression
from sklearn.metrics import mean_squared_error

lr = LinearRegression(fit_intercept = True) #
Fit intercept is the "b" parameter (y = b + mx)
lr.fit(X_train, y_train)
```

Screenshot:

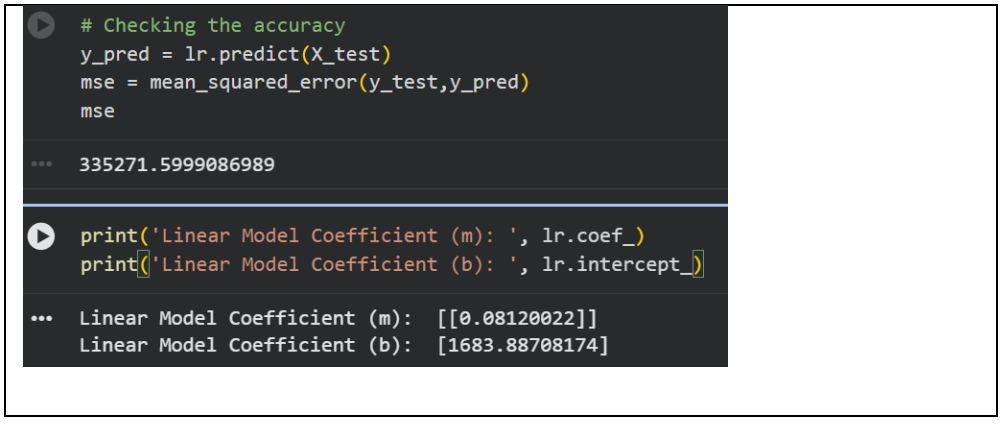


Kode program untuk mengecek akurasi serta mencetak nilai gradien dan intersep:

```
# Checking the accuracy
y_pred = lr.predict(X_test)
mse = mean_squared_error(y_test,y_pred)
mse

print('Linear Model Coefficient (m): ',
lr.coef_)
print('Linear Model Coefficient (b): ',
lr.intercept_)
```

Screenshot:



14. Visualisasi hasil pelatihan model regresi linear.

Kode progra:

```
# Plotting the TRAIN DATA
plt.figure(figsize=(10, 6))
plt.scatter(X_train, y_train, color='gray',
label='Data Asli')
```

```
plt.plot(X_train, lr.predict(X_train),
color='red', linewidth=2, label='Garis Regresi')
plt.ylabel('Calories Burned (kcal)')
plt.xlabel('Steps')
plt.title('Garis Regresi Linear: Steps vs.
Calories Burned (Training Data)')
plt.legend()
plt.show()
```

Screenshot:



15. Simpan model dalam bentuk file pickle yaitu binari python dan simpan di folder apliaksi streamlit.

    Kode program:

```
import joblib
from google.colab import files

joblib.dump(lr,
"linear_regression_stepstocal.pkl")

files.download("linear_regression_stepstocal.pkl
")
```

Screenshot:

```
import joblib
from google.colab import files

joblib.dump(lr, "linear_regression_stepstocal.pkl")

files.download("linear_regression_stepstocal.pkl")
```

16. Di folder streamlit, buat file python baru.

Kode program:

```
import streamlit as st
import sklearn
import joblib
import os
import numpy as np

# Function untuk meload model
def load_prediction_model(model_file):
    loaded_model =
joblib.load(open(os.path.join(model_file), "rb"))
    return loaded_model

def main():
    """Simple Linear Regression - Steps to Calories"""
    st.title("Prediksi Kalori Terbakar")
    html_templ = """
    <div style="background-
color:#4CAF50;padding:10px;border-radius:5px;">
    <h3 style="color:white;text-align:center;">Prediksi
Pembakaran Kalori Menggunakan Regresi Linier</h3>
    </div>
    <br>
    """
    st.markdown(html_templ, unsafe_allow_html=True)
    activity = ["Prediksi Kalori", "Apa itu Regresi?"]
    choice = st.sidebar.selectbox("Menu", activity)

    #Prediksi Kalori
    if choice == 'Prediksi Kalori':
        st.subheader("Hitung Estimasi Pembakaran Kalori")
        # Step input slider (range: 500 to 35,000 steps)
        steps = st.slider("Berapa jumlah langkah
harianmu?", min_value=500, max_value=35000, value=500,
step=1)
        if st.button("Proses"):
            # Load model
            regressor =
load_prediction_model("./linear_regression_stepstocal.pkl"
)
            # Ubah input ke 2D array [[steps]]
```

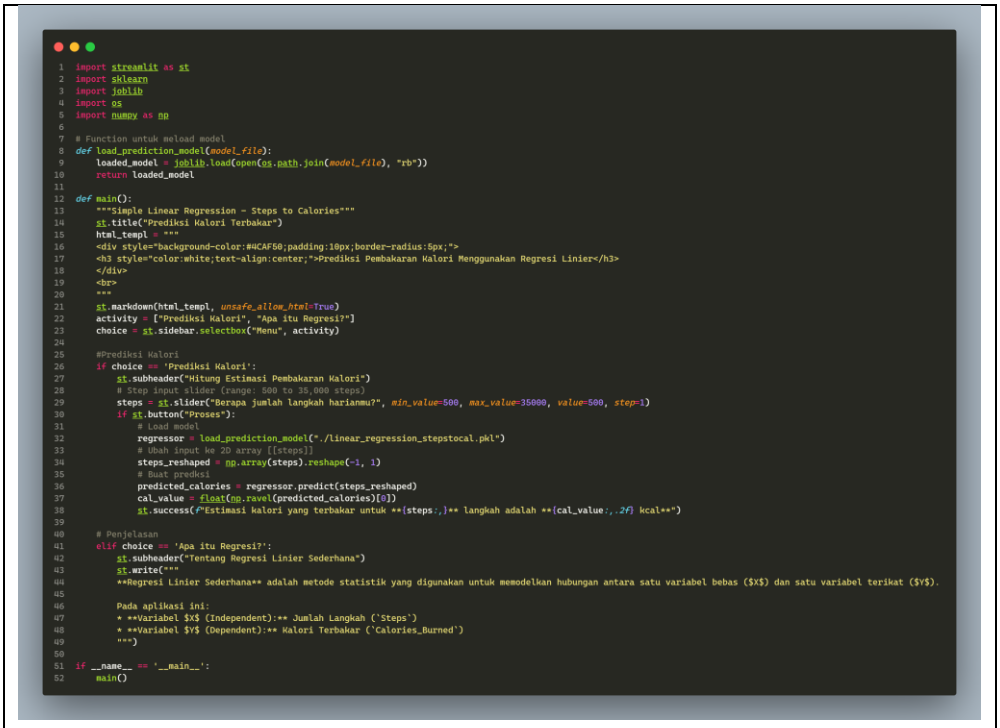
```
            steps_reshaped = np.array(steps).reshape(-1,
1)
            # Buat predksi
            predicted_calories =
regressor.predict(steps_reshaped)
            cal_value =
float(np.ravel(predicted_calories)[0])
            st.success(f"Estimasi kalori yang terbakar
untuk **{steps:,}** langkah adalah **{cal_value:,.2f}
kcal**")

    # Penjelasan
    elif choice == 'Apa itu Regresi?':
        st.subheader("Tentang Regresi Linier Sederhana")
        st.write("""
        **Regresi Linier Sederhana** adalah metode
statistik yang digunakan untuk memodelkan hubungan antara
satu variabel bebas ($X$) dan satu variabel terikat ($Y$).

        Pada aplikasi ini:
        * **Variabel $X$ (Independent):** Jumlah Langkah
(`Steps`)
        * **Variabel $Y$ (Dependent):** Kalori Terbakar
(`Calories_Burned`)
        """)

if __name__ == '__main__':
    main()
```

Screenshot:



```python
import streamlit as st
import sklearn
import joblib
import os
import numpy as np

# Function untuk reload model
def load_prediction_model(model_file):
    loaded_model = joblib.load(open(os.path.join(model_file), "rb"))
    return loaded_model

def main():
    """Simple Linear Regression - Steps to Calories"""
    st.title("Prediksi Kalori Terbakar")
    html_templ = """
    <div style="background-color:#4CAF50;padding:10px;border-radius:5px;">
    <h3 style="color:white;text-align:center;">Prediksi Pembakaran Kalori Menggunakan Regresi Linier</h3>
    </div>
    <br>
    """
    st.markdown(html_templ, unsafe_allow_html=True)
    activity = ["Prediksi Kalori", "Apa itu Regresi?"]
    choice = st.sidebar.selectbox("Menu", activity)

    #Prediksi Kalori
    if choice == 'Prediksi Kalori':
        st.subheader("Hitung Estimasi Pembakaran Kalori")
        # Step input slider (range: 500 to 35,000 steps)
        steps = st.slider("Berapa jumlah langkah harianmu?", min_value=500, max_value=35000, value=500, step=1)
        if st.button("Proses"):
            # Load model
            regressor = load_prediction_model("./linear_regression_stepstocal.pkl")
            # Ubah input ke 2D array [[steps]]
            steps_reshaped = np.array(steps).reshape(-1, 1)
            # Buat prediksi
            predicted_calories = regressor.predict(steps_reshaped)
            cal_value = float(np.ravel(predicted_calories)[0])
            st.success(f"Estimasi kalori yang terbakar untuk **{steps:,}** langkah adalah **{cal_value:,.2f} kcal**")

    # Penjelasan
    elif choice == 'Apa itu Regresi?':
        st.subheader("Tentang Regresi Linier Sederhana")
        st.write("""
        **Regresi Linier Sederhana** adalah metode statistik yang digunakan untuk memodelkan hubungan antara satu variabel bebas ($X$) dan satu variabel terikat ($Y$).

        Pada aplikasi ini:
        * **Variabel $X$ (Independent):** Jumlah Langkah (`Steps`)
        * **Variabel $Y$ (Dependent):** Kalori Terbakar (`Calories_Burned`)
        """)

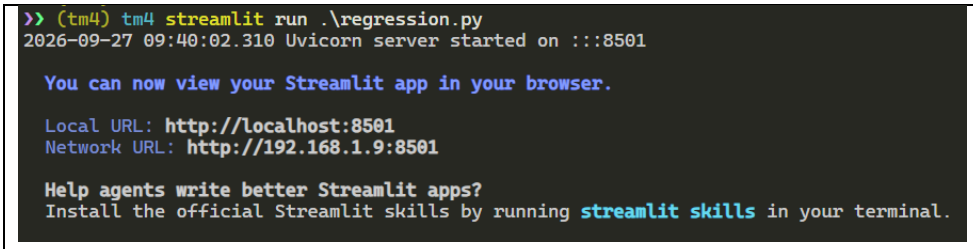
if __name__ == '__main__':
    main()
```

17. Menjalankan aplikasi Streamlit pada file regression.py

    Perintah di Powershell:

```
streamlit run .\regression.py
```

Screenshot:



```
>> (tm4) tm4 streamlit run .\regression.py
2026-09-27 09:40:02.310 Uvicorn server started on :::8501

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://192.168.1.9:8501

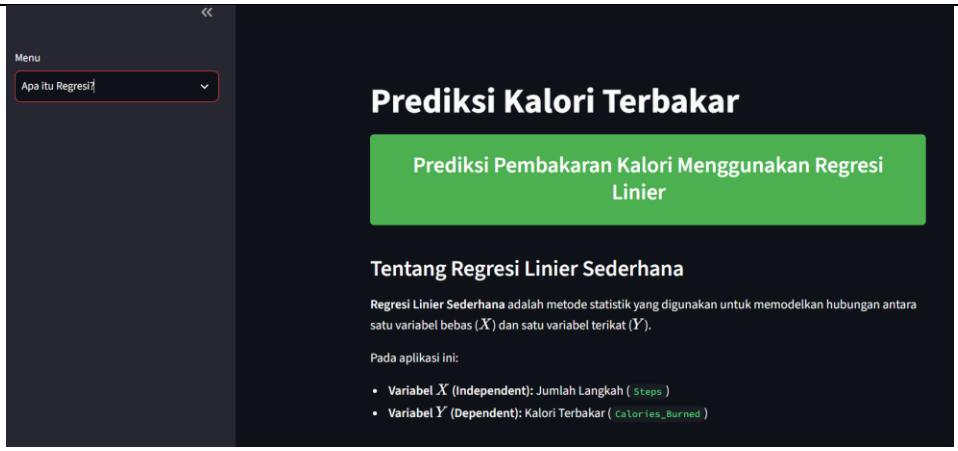
  Help agents write better Streamlit apps?
  Install the official Streamlit skills by running streamlit skills in your terminal.
```

18. Membuka URL dari terminal: http://localhost:8501/

    Screenshot:



Screenshot:



C. Kendala yang Dialami

Tidak ada kendala yang dialami. Praktikum berjalan dengan lancar.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, aplikasi web Streamlit berhasil dibuat untuk memprediksi estimasi pembakaran kalori harian berdasarkan jumlah langkah kaki (*Steps*). Pemodelan regresi linear sederhana diimplementasikan menggunakan library Scikit-Learn dengan mengolah dataset aktivitas harian. Data terlebih dahulu difilter untuk mengeliminasi anomali sensor serta memfokuskan analisis pada aktivitas

berjalan aktif. Model prediksi yang telah dilatih kemudian dieksport ke dalam format Pickle (.pkl) dan diintegrasikan secara interaktif ke dalam antarmuka Streamlit. Hasil prediksi kemudian disajikan secara terstruktur dalam bentuk pesan estimasi kalori (*success metric*) serta dilengkapi halaman penjelasan edukatif mengenai konsep regresi linear.